# Лекція №21 (2 години). Основи Node.js та Express для веб-розробки.

## План лекції

1. Що таке Node.js та движок V8.
2. Робота з файловою системою та модулями (CommonJS vs ES Modules).
3. Що таке Backend та архітектура клієнт-сервер.
4. Введення в Express.js: найпопулярніший фреймворк для Node.js.
5. Створення базового HTTP-сервера.
6. Роутинг (Маршрутизація) в Express (GET, POST запити).
7. Middleware (Проміжне програмне забезпечення).
8. Створення найпростішого REST API.

## Перелік умовних скорочень

- **API** (Application Programming Interface) — інтерфейс прикладного програмування.
- **REST** (Representational State Transfer) — архітектурний стиль для API.
- **NPM** (Node Package Manager) — менеджер пакетів для JavaScript.
- **HTTP** (HyperText Transfer Protocol) — протокол передачі гіпертексту.

## Вступ

Довгий час JavaScript був мовою, яка жила виключно в одному місці — у браузері користувача. Він вчився лише керувати кнопками, малювати анімації і валідувати форми. Якщо вам потрібен був Сервер (Backend), який міг би зберігати дані в базу, авторизувати користувачів та обробляти платежі, вам доводилося вчити зовсім іншу мову: PHP, Python, Java або Ruby.

Але у 2009 році світ перекинула технологія **Node.js**. Вона дозволила "витягнути" рушій JavaScript з браузера і встановити його прямо на серверну операційну систему (Windows, Linux, macOS). Раптом JavaScript отримав можливість працювати з комп'ютером на низькому рівні: читати файли на диску, відкривати мережеві порти, звертатися до баз даних.

Це створило еру **Full-stack JavaScript розробки** (MERN-стек). Тепер розробник міг написати весь проект від початку до кінця на одній мові — JS.

Мета цієї лекції — зрозуміти, що таке серверний JS, навчитися піднімати власний сервер та створювати власне API за допомогою фреймворку Express.

## 1. Що таке Node.js?

**Node.js** — це середовище виконання (runtime environment). Воно побудоване на основі двигуна **V8**, який розробляє компанія Google (саме на ньому працює браузер Google Chrome).

Node.js написаний на мові C++. Він бере ваш JS-код, компілює його у надшвидкий машинний код і дозволяє йому виконуватися прямо в операційній системі.

**Головні особливості Node.js:**

1. **Асинхронність та неблокуючий I/O (Input/Output):** Node.js створений для обробки тисяч одночасних підключень. Він працює в одному потоці (як і браузерний JS), використовуючи знайомий нам механізм Event Loop. Коли сервер звертається до повільної бази даних, він не "зависає", чекаючи на відповідь, а миттєво перемикається на обслуговування наступного клієнта.
2. **Npm (Node Package Manager):** Найбільша у світі екосистема відкритих бібліотек. Будь-яка задача (парсинг файлів, генерація PDF) вже кимось вирішена і доступна для встановлення через термінал.

## 2. Робота з файловою системою та модулями

Оскільки Node.js немає доступу до `document` чи `window` (бо немає браузера і DOM!), у нього є свої вбудовані "суперсили" — глобальні модулі (наприклад `fs` - File System, `http`, `path`).

Приклад читання текстового файлу з жорсткого диска комп'ютера:

```
// Старий формат імпорту (CommonJS), що досі часто використовується в Node.js
const fs = require("fs");

// Читаємо файл асинхронно
fs.readFile("message.txt", "utf8", (err, data) => {
  if (err) {
    console.error("Помилка читання файлу:", err);
    return;
  }
  console.log("Вміст файлу: ", data);
});
```

Останні версії Node.js підтримують і сучасний синтаксис `import/export` (ES Modules), до якого ми звикли у React. Для цього в конфігураційному файлі проекту `package.json` потрібно вказати `"type": "module"`.

## 3. Що таке Backend та сервер?

**Сервер** — це комп'ютер, який працює 24/7, підключений до Інтернету та постійно "слухає" певний порт (наприклад, 80 для HTTP, або 443 для HTTPS). Його задача: прийняти Запит (Request) від Клієнта (браузера або мобільного додатку), зрозуміти, що хоче Клієнт, зробити розрахунки і повернути Відповідь (Response).

Node.js вміє створювати сервери за допомогою вбудованого модуля `http`, але код виходить дуже довгим і низькорівневим (доводиться вручну перевіряти URL-рядок, вирізати потрібні шматки URL, розбирати кожен HTTP-заголовок).

Щоб не винаходити велосипед, 95% розробників використовують фреймворк абстракції — **Express.js**.

## 4. Введення в Express.js

**Express.js** — це мінімалістичний, гнучкий веб-фреймворк для Node.js. Завдяки йому написати повноцінний сервер, який може віддавати JSON та обробляти запити користувачів, можна всього в 10 рядків коду. Він бере на себе всю важку роботу з маршрутизацією та обробкою помилок.

**Ініціалізація проекту:**

```
npm init -y        # Створює package.json (паспорт проекту)
npm install express # Встановлює фреймворк
```

## 5. Створення базового сервера

Створимо файл `index.js`. Ось вичерпний код для створення справжнього HTTP-сервера:

```
// 1. Підключаємо Express
const express = require("express");

// 2. Створюємо "додаток" (екземпляр сервера). Його традиційно називають app.
const app = express();

const PORT = 3000; // На якому порту буде "жити" сервер
```

```
// 3. Створюємо перший маршрут (Route).
// Коли хтось зробить GET-запит на головну сторінку ('/'), запуститься ця функція
app.get("/", (req, res) => {
  // res.send - це зручна команда Express для відправки текстової відповіді
  res.send("Привіт, Світ! Це мій перший бекенд.");
});

// 4. Запускаємо сервер: він починає "слухати" порт
app.listen(PORT, () => {
  console.log(`Сервер успішно запущено на http://localhost:${PORT}`);
});
```

Тепер, якщо ми відкриємо термінал і виконаємо `node index.js`, сервер "оживе". Ми зможемо відкрити вкладку браузера на адресі `http://localhost:3000` і побачити наш текст.

## 6. Роутинг (Маршрутизація) в Express

Звісно, сервер зазвичай має обслуговувати не один URL, а десятки різних шляхів. В Express ми прив'язуємо Логіку до **HTTP Методу** ( `GET`, `POST`, `DELETE` ) та **Шляху** ( `/users`, `/products` ).

Як ми згадували в лекції про REST API:

- `app.get()` — для читання даних.
- `app.post()` — для додавання нових даних (наприклад, з форми реєстрації).
- `app.delete()` — для видалення.

Кожен такий обробник завжди приймає два важливі аргументи: **`req`** (Request/Запит) та **`res`** (Response/Відповідь).

- В об'єкті **`req`** лежать всі дані, з якими до нас прийшов клієнт (наприклад, яка в нього IP адреса, який у нього токен авторизації, які додаткові параметри він передав у URL).
- На об'єкті **`res`** лежать функції, якими ми користуємося, щоб відправити клієнту результат і завершити транзакцію (наприклад `res.json()`, `res.status(404)` ).

**Створення найпростішого REST API (Повертаємо JSON):**

```
// Імітуємо базу даних у пам'яті
const users = [
  { id: 1, name: "Олександр", role: "admin" },
  { id: 2, name: "Марія", role: "user" },
```

```
];

// Коли React-додаток (або Postman) звернеться до localhost:3000/api/users
app.get("/api/users", (req, res) => {
  // Замість res.send() (який віддає текст/html), використовуємо res.json()
  // Він автоматично перетворить масив JS у формат JSON і відправить клієнту
  res.json(users);
});
```

**Динамічні URL (як у React Router):** Як знайти одного конкретного користувача?

```
// Двокрапка :id перетворює цю частину шляху на змінну (параметр)
app.get("/api/users/:id", (req, res) => {
  // Всі такі змінні лежать в об'єкті req.params
  const userId = Number(req.params.id);

  // Шукаємо користувача масиві
  const user = users.find((u) => u.id === userId);

  if (!user) {
    // Якщо не знайшли - ставимо статус 404 (Not Found)
    return res.status(404).json({ error: "Користувача не знайдено" });
  }

  // Все добре (статус 200 ставиться автоматично), віддаємо юзера
  res.json(user);
});
```

## 7. Middleware (Проміжне ПЗ)

**Middleware** (Мідлвари) — це серце філософії Express. Мілвар — це функція, яка має доступ до об'єкта запиту ( `req` ) і об'єкта відповіді ( `res` ), і запускається **між** тим моментом, як запит прийшов на сервер, і тим моментом, як спрацює ваш фінальний роут (наприклад `app.get('/users')` ).

Нащо вони потрібні? Для обробки "чорнової роботи", що повторюється для ВСІХ або декількох вибраних маршрутів. Приклади Middleware:

- **Логування:** Записувати в консоль кожен запит ( `"Отримано запит GET /api/users о 14:00"` ).
- **Парсинг:** Брати JSON-текст, який надіслав клієнт, і перетворювати назад в JS-об'єкт.

- **Авторизація:** Перевіряти, чи є у клієнта секретний токен; якщо ні — заборонити доступ і послати помилку 401 Unauthorized, навіть не доводячи запит до бази даних.

**Глобальний Мідлвар логування (Виконується для будь-якого запату):**

```
// Слово "use" означає "використати мідлвар"
app.use((req, res, next) => {
  console.log(`[ЛЗГ] Новий запит: Метод ${req.method} на адресу ${req.url}`);

  // next() - КРИТИЧНО ВАЖЛИВА функція. Вона каже Express-у:
  // "Я закінчив свою проміжну роботу, передаю естафету наступній функції (вашому
роуту)".
  // Якщо не викликати next() - клієнт зависне (буде чекати вічно).
  next();
});
```

**Вбудований Мідлвар для `POST`-запитів:** За замовчуванням Express не вміє читати JSON, який присилає йому React-додаток при відправці форм. Щоб навчити його, достатньо одного рядка (це вбудований мідлвар):

```
app.use(express.json()); // Вчить Express збирати JSON з тіла запиту і класти його
в req.body
```

Тепер ми можемо обробляти `POST` запити (створення):

```
app.post("/api/users", (req, res) => {
  // В req.body лежать нові дані від клієнта (наприклад { name: "Іван", role:
"user" })
  const newUser = req.body;

  // Генеруємо новий ID для бази
  newUser.id = users.length + 1;
  users.push(newUser);

  // Відповідаємо статусом 201 (Created)
  res.status(201).json(newUser);
});
```

## Висновки

1. **Node.js** — це середовище, яке дозволило використовувати JavaScript на бекенді (сервері), взаємодіяти з операційною системою (ОС) та відкривати мережеві порти.
2. Головна перевага Node.js — асинхронна (неблокуюча) архітектура Event Loop, що дозволяє одному серверу ефективно одночасно обслуговувати тисячі різних користувачів (багато клієнтів), економлячи пам'ять.
3. Процес створення "голого" сервера є громіздким, тому індустрія перейшла на **Express.js** — мінімалістичний і швидкий веб-фреймворк для легкої маршрутизації шляхів.
4. Розробка REST API в Express зводиться до декларації маршрутів, пов'язаних з HTTP-методами: `app.get()`, `app.post()`, `app.delete()`.
5. Гнучкість платформи будується навколо **Об'єкта Запиту** ( `req` — містить урл-параметри, тіло повідомлення, заголовки) та **Об'єкта Відповіді** ( `res` — керує HTTP-статусами та відправкою даних через команду `res.json()` ).
6. **Middleware** (проміжне програмне забезпечення) — це набір своєрідних "фільтрів", через які пропускається кожен клієнтський запит, перш ніж досягти маршруту логіки. Використовуються для перевірки авторизації чи автоматичного розпакування JSON-об'єктів (через `app.use(express.json())` ).

## Джерела

1. Node.js Official Documentation — офіційна документація Node.js українською.
2. Express.js Getting Started — посібник зі встановлення та створення "Hello, World" сервера за 5 хвилин.
3. MDN: Express Web Framework (Node.js/JavaScript) — комплексний курс по створенню бекенду від авторів Firefox.

## Запитання для самоперевірки

1. Чому Node.js став революцією і звідки він взяв свій головний "двигун" (Engine) для обробки коду?
2. Поясніть принцип "неблокуючого вводу/виводу" (Non-blocking I/O). Що робить сервер на Node.js, поки чекає інформацію від повільної бази даних?
3. Що таке `npm` і яку роль цей концепт відіграє в екосистемі Node?
4. Для чого Frontend-розробники підключають фреймворк `Express` замість того, щоб масувати обробку запитів голим Node.js модулем "http"?
5. Що таке Middleware (Мідлвари) у філософії Express? Навіщо вони потрібні та яку функцію обов'язково треба викликати всередині мідлвару, щоб код працював далі (згадайте функцію під кодом `next` )?
6. Ви отримали клієнтський `GET` запит вигляду `/users/99`. Як називається об'єкт (його поле), всередині якого лежить число 99, і як з ним працює Express-навігація ( `req.xxx...` )?

7. Ваш сервер має прочитати повідомлення з форми (POST запит) від користувача і записати у таблицю бази. Який вбудований мідлвар треба додати у верхній частині файлу конфігурації `app.js`, щоб об'єкт `req.body` не був `undefined`?